Practical - 5

Government College of Engineering, Jalgaon (An Autonomous Institute of Govt. of Maharashtra) Department of Computer Engineering	
Name:	
PRN:	**Batch:**
Class:	**Semester:**
Course:	**Subject Teacher:**
Date of Performance:	**Date of Completion:**

Aim : Design a distributed application which consist of a server and client using threads.

Theory :

PROCESS:

In a conventional centralized operating system process management deals with mechanisms and policies for sharing the processor the system among all processes .Similarly ,in a distributed operating system ,the main goal-of process management to make the best possible use of the processing resources of the entire system by sharing the among all processes. Three important concepts are used in distributed operating systems to achieve this goal:

1. Processor allocation:

   Deals with the process of deciding which process should be assigned to which processor.
2. Process migration:

   Deals with the movement of a process from its current location of the processor to which has been assigned.Deals with fine-grained parallelism for better utilization of the processing capability of the system The processor all location concept ready been described in the previous chapter on resource management. Therefore, this chapter presents a description of the process migration and threads concepts.

THREAD:

A thread is a flow of execution through the process code, with its own program counter that keeps track of which instruction to execute next, system registers which hold its current working variables, and a stack which contains the execution history.A thread shares with its peer threads few information like code segment, data segment and open files.When one thread alters a code segment memory item, all other threads see that.

A thread is also called a lightweight process. Threads provide a way to improve application performance through parallelism. Threads represent a software approach to improving performance of operating system by reducing the overhead thread is equivalent to a classical process.

ADVANTAGES OF THREAD

1. Threads minimize the context switching time.
2. Use of threads provides concurrency within a process.
3. Efficient communication.
4. It is more economical to create and context switch threads.
5. Threads allow utilization of multiprocessor architectures to a greater scale and efficiency.

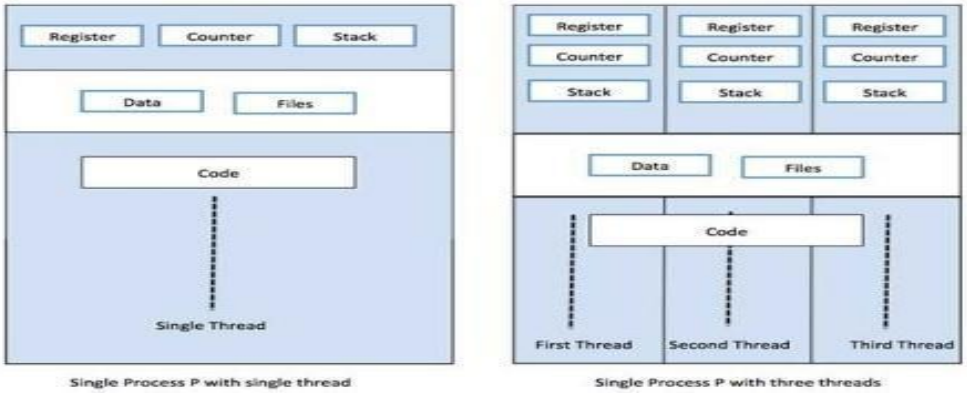


Figure 1: thread concept with Process

MULTITHREADING:

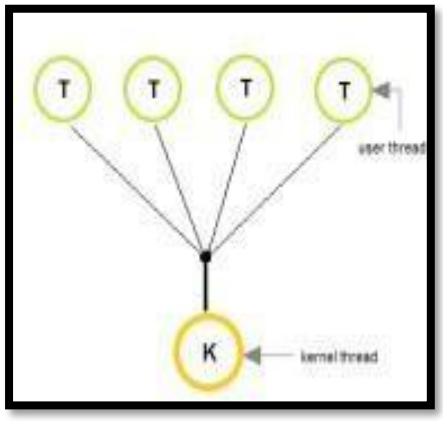
Multithreading is the ability of a program or an operating system process to manage its use by more than one user at a time and to even manage multiple requests by the same user without having to have multiple copies of the programming running in the computer. Each user request for a program or system service (and here a user can also be another program) is kept track of as a thread with a separate identity. As programs

work on behalf of the initial request for that thread and are interrupted by other requests, the status of work on behalf of that thread is kept track of until the work is completed.
The user threads must be mapped to kernel threads, by one of the following strategies:

1. Many to One Model
2. One to One Model
3. Many to Many Model

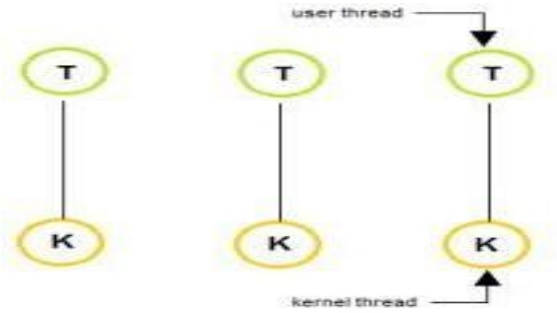
1. Many to One Model

1. As shown in figure 2, In the **many to one** model, many user-level threads are all mapped onto a single kernel thread.
2. Thread management is handled by the thread library in user space, which is efficient in nature.



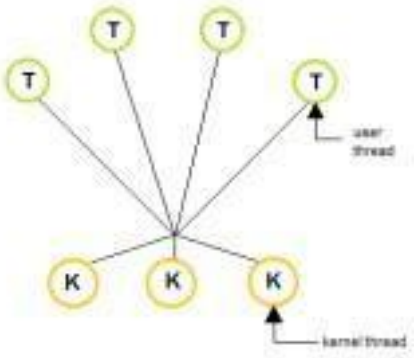
2.One to One Model:

1. As shown in figure 3 The **one to one** model creates a separate kernel thread to handle each and every user thread.
2. Most implementations of this model place a limit on how many threads can be created.



3 .Many to Many Model

1. The many to many model multiplexes any number of user threads onto an equal or smaller
2. Linux and Windows from 95 to XP implement the one-to-one model for threads



Difference between Process and Thread:

S.N.	Process	Thread
1	Process is heavy weight or resource intensive.	Thread is light weight, taking lesser resources than a process.
2	Process switching needs interaction with operating system.	Thread switching does not need to interact with operating system.
3	In multiple processing environments, each process executes the same code but has its own memory and file resources.	All threads can share same set of open files, child processes.
4	If one process is blocked, then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, a second thread in the same task can run.
5	Multiple processes without using threads use more resources.	Multiple threaded processes use fewer resources.
6	In multiple processes each process operates independently of the others.	One thread can read, write or change another thread's data.

ALGORITHM:

Steps

1.Client Program:

1. Initialise input and output stream.
2. Open socket on given port.
3. Open output and input stream.
4. If everything has been initialized then we want to write some data to the socket we have opened a connection to on the given port
5. Close input and output stream.
6. Close the socket

2.Server Program:

1. Calling constructor of server final and initializing in object(server) of serverfinal.
2. Calling startserver function
3. Declare a server socket and a client socket for the server
4. Declare the number of connections
5. Try to open a server socket on the given port(Note that we can't choose a port less than 1024 if we are not)
6. Whenever a connection is received, start a new thread to process the connection and wait for the next connection.
7. Close the client connection.

Code :

Server.py-

```
import socket
import threading
import sys
# Server configuration
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12345
BUFFER_SIZE = 1024

isServer = 0
# Function to handle each client connection
def handle_client(client_socket, address):
    global isServer
    print(f"[+] Connected with {address}")

    while True:
        # Receive data from client
        data = client_socket.recv(BUFFER_SIZE)
        print("Data here ", data)
        if not data:
            break
        if data == b'STOP_SERVER':
            isServer = 1
            print("Here is data", data)
            print("Server is stopped by client\' input.")
            sys.exit(0)
        # Echo back the received data
        client_socket.sendall(data)

    # Close connection with client
    client_socket.close()
    print(f"[-] Connection with {address} closed")

# Function to start the server
def start_server():
    global isServer
    # Create a TCP socket
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    # Bind the socket to the server address
    server_socket.bind((SERVER_HOST, SERVER_PORT))

    # Listen for incoming connections
    server_socket.listen(5)
    print(f"[*] Listening on {SERVER_HOST}:{SERVER_PORT}")

    while not isServer:
```

```
        # Accept incoming connection
        client_socket, address = server_socket.accept()

        # Create a new thread to handle the client
        client_thread = threading.Thread(target=handle_client, args=(client_socket, address))
        client_thread.start()

if __name__ == "__main__":
    start_server()
```

Client.py-

```
import socket

# Server configuration
SERVER_HOST = '127.0.0.1'
SERVER_PORT = 12345
BUFFER_SIZE = 1024

def start_client():
    # Create a TCP socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        # Connect to the server
        client_socket.connect((SERVER_HOST, SERVER_PORT))

        while True:
            # Ask for user input
            choice = input("Enter 0 to stop connection, 1 to stop server: ")
            if choice == '0':
                break
            elif choice == '1':
                # Send a special message to the server to stop
                client_socket.sendall(b"STOP_SERVER")
                break
            else:
                print("Invalid choice. Enter 0 or 1.")

    finally:
        # Close the connection
        client_socket.close()

if __name__ == "__main__":
    start_client()
```

Output:

Server.py

```
D:\Me\College\DSL\5_Threads>java Server_Threads
Server started on port 6789
Waiting for clients...
Client #1 connected: Socket[addr=/127.0.0.1,port=63434,localport=6789]
Client #2 connected: Socket[addr=/127.0.0.1,port=55289,localport=6789]
Client #1 sent: 0
Client #1 disconnected
Client #2 sent: -1
Client #2 disconnected
Shutting down server...
```

Client.py

```
D:\Me\College\DSL\5_Threads>java Client_Threads
Connected to server.
Enter number (0 = exit, -1 = stop server): 0
Connection closed.
```

Conclusion:
In this Practical, We learnt about development of distributed application consisting server and client using threads

Sign of Teacher
Mrs. Archana Chitte.